



## Security and Governance

Google Cloud

Proprietary & Confidential

1

Welcome to Module 5: Security and Governance.

So far in the course we've discussed how to deploy agents, observe their behavior, and evaluate their quality.

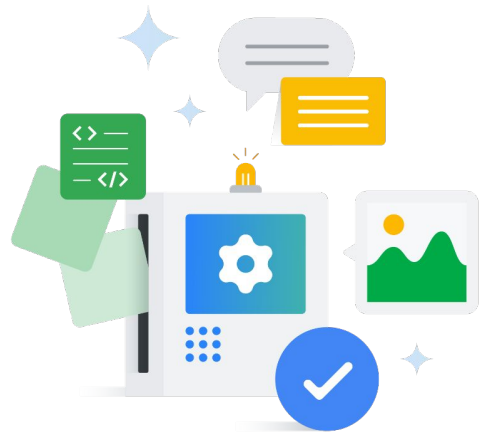
However, none of that matters if the system is not secure.

Agent systems introduce new security risks because they interact with users, external tools, APIs, and data sources.

In this module, we'll explore how to secure both model interactions and agent infrastructure using Google Cloud services.

## In this module, you learn to ...

- 01 Secure model inputs and outputs with Model Armor
- 02 Protect sensitive data using Sensitive Data Protection with Model Armor
- 03 Secure the connection between a user and an agent

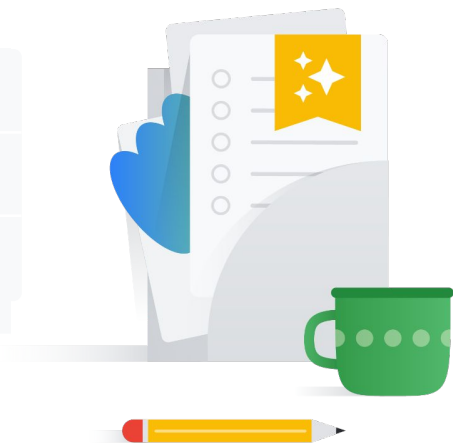


In this module, you'll learn three major capabilities.

- First, how to secure model inputs and outputs using Model Armor.
- Second, how to protect sensitive data using Sensitive Data Protection, also known as the DLP service.
- Third, how to secure the connection between users and agents using identity and access management patterns.

# Topics

- |    |                                                                           |
|----|---------------------------------------------------------------------------|
| 01 | Model and context security                                                |
| 02 | Agent access                                                              |
| 03 | Lab: Enhancing AI Security with Model Armor and Sensitive Data Protection |



This module is divided into three sections.

- First we'll cover model and context security, which focuses on protecting the model interaction layer.
- Second we'll discuss agent access, which focuses on authentication and authorization.
- Finally, we'll complete a hands-on lab that demonstrates how to enforce safety policies using Model Armor and Sensitive Data Protection.

# Challenges for security with agentic applications

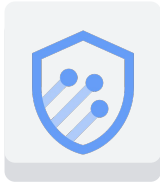
| Malicious files and unsafe URLs                                                                                                                                                                                  | Prompt injection and jailbreak attacks                                                                                                                                                 | Sensitive data exposure                                | Responsible AI safety risk                                      |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|-----------------------------------------------------------------|
| <p>Embedded instructions in files to manipulate reasoning and trigger unauthorized tool execution.</p> <p>Disguising malicious URLs to look legitimate, causing phishing attacks, malware distribution, etc.</p> | <p>Including commands within the prompt to trick an AI model to ignore its instructions</p> <p>Bypassing the safety protocols and ethical guidelines that are built into the model</p> | <p>Data leakage going into and out of AI workloads</p> | <p>Agents can generate biased, unsafe, or harmful decisions</p> |

Agentic applications introduce several new security challenges.

- One major risk is prompt injection, where malicious instructions attempt to override the model's intended behavior.
- Another risk is malicious files or URLs, which may contain hidden instructions designed to manipulate the agent.
- There is also the risk of sensitive data exposure, where confidential information could appear in prompts or responses.
- Finally, agents may introduce Responsible AI risks, such as generating biased or harmful content.

These risks require both input filtering and output filtering mechanisms.

# Model Armor



Model Armor



Designed to enhance the security and safety of AI applications



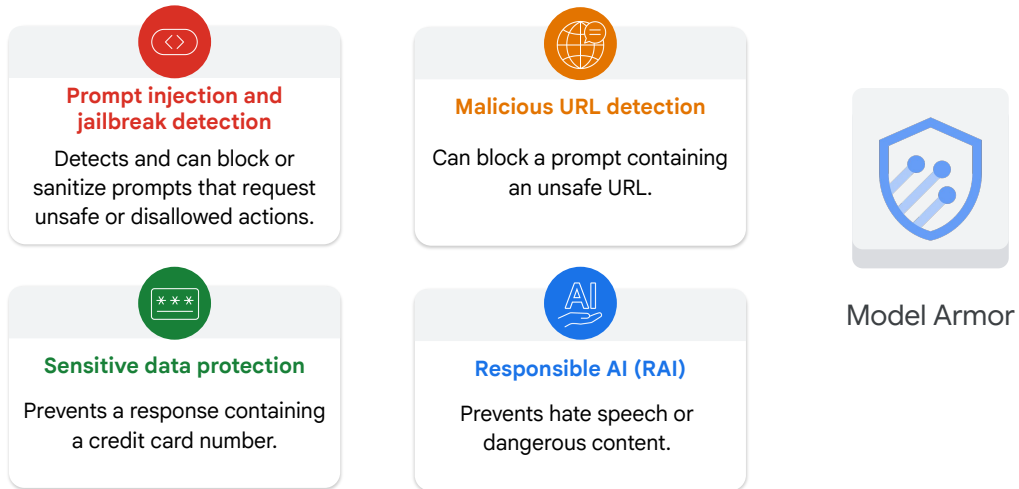
Analyzes and flags prompts and responses for potential risks



Offers a variety of filters to help provide safe and secure AI models

Model Armor is a Google Cloud service designed to enhance the security and safety of your AI applications. It works by proactively screening LLM prompts and responses, protecting against various risks and ensuring responsible AI practices. Whether you are deploying AI in Google Cloud or other cloud providers, Model Armor can help you prevent malicious input, verify content safety, protect sensitive data, maintain compliance, and enforce your AI safety and security policies consistently across your AI applications.

# Model Armor capabilities

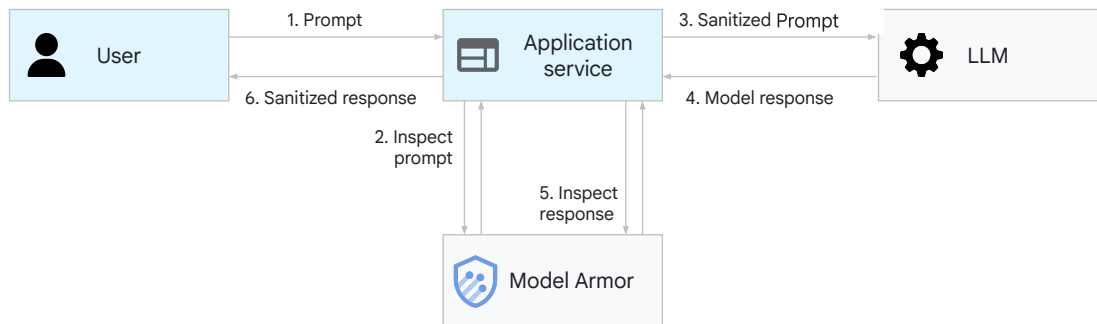


Model Armor provides several types of detection capabilities.

- It can detect prompt injection and jailbreak attempts.
- It can detect malicious URLs embedded in prompts.
- It can identify sensitive data, such as credit card numbers or credentials.
- And it can enforce Responsible AI safety filters, preventing harmful or unsafe outputs.

These capabilities help protect both the model and downstream systems.

# Model Armor architecture



Model Armor can help to mitigate threats like malicious URLs, prompt injection, jailbreaking, sensitive data leaks, and improper output handling.

The architecture places Model Armor between the application and the model.

When a user sends a prompt, Model Armor inspects it first.

If the prompt passes validation, the sanitized prompt is forwarded to the model.

The model produces a response, which is again inspected by Model Armor before returning to the user.

This ensures both input and output are validated.

# Model Armor's floors and templates

## Floor settings

Establish the minimum security requirements that templates are required to meet.

## Templates

Dial in exactly how Model Armor examines prompts and responses.

Model Armor offers this tailored protection through two core features: floor settings and templates.

Floor settings are the bedrock of security. They establish the bare minimum security requirements that all your custom configurations within the template must follow.

The template is like your control panel, where you dial in exactly how Model Armor examines prompts and responses.

# Floor settings

Floor settings are hierarchical, can be set at the:

- Project
- Folder
- Organization

▼ gcp-security.com

▼ Team A

my-cool-agent

← Configure floor settings

### Floor settings

Floor settings ensure all Model Armor templates have minimum thresholds for detections. In addition, you can sanitize requests and responses across all Gemini models in Vertex AI and Google managed MCP servers.

#### Configuration options

☐ Inherit parent's floor settings  
Inherit floor settings from the closest parent level. If no settings are enabled there, it will continue to inherit from higher levels in the resource hierarchy.

☒ Custom  
Define project-specific floor settings, overriding any inherited values.

☐ Disable  
Disable any inherited floor settings.

### Configure floor settings

#### Detections

☐ Malicious URL detection  
Identifies web addresses (URLs) that are designed to harm users or systems. These URLs might lead to phishing sites, malware downloads, or other cyberattacks.

☒ Prompt injection and jailbreak detection  
Prompt injection is when a malicious actor tries to insert malicious content into a prompt. A jailbreak attempt is when a malicious actor tries to break out of the model's safety controls. Both can make the AI ignore its usual instructions, reveal sensitive information, bypass an AI model's safeguards, or make it perform actions it wasn't designed to do.

Confidence level  
Medium and above  
For stricter enforcement, set confidence level to Low and above. This will detect most content that is likely to be a prompt injection and/or jailbreak attempt.

☐ Sensitive data protection  
Detects sensitive data and helps prevent its accidental exposure from attacks like prompt injection.

#### Responsible AI

Confidence level represents how type. For stricter enforcement, set most content that falls into a category.

#### Enforcement

Choose where these floor settings will be applied and the actions to take when a request or response does not meet the floor setting threshold. [View details](#)

☒ Model Armor - Template create and update  
Check every new and modified template in this project to ensure they meet the minimum floor settings.

☐ Google Managed MCP Servers  
Check if requests sent to/from Google MCP servers meet the floor settings thresholds. MCP server calls to Model Armor might violate Data Residency (DRZ) processing depending on the services used. See [Model Armor supported products](#) for more details.

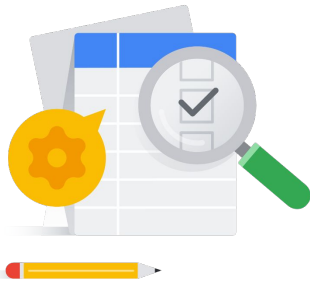
☒ Agent Platform  
Check if requests sent to/from Gemini models meet the floor settings thresholds.

☒ Inspect only  
☐ Inspect and block violations

Floor settings are hierarchical.

They can be applied at the organization level, the folder level, or the project level.

This hierarchy allows security teams to enforce baseline protections across all applications while allowing individual teams to add stricter rules when needed.



## Confidence levels

Confidence levels are the sensitivity instructions given to Model Armor.

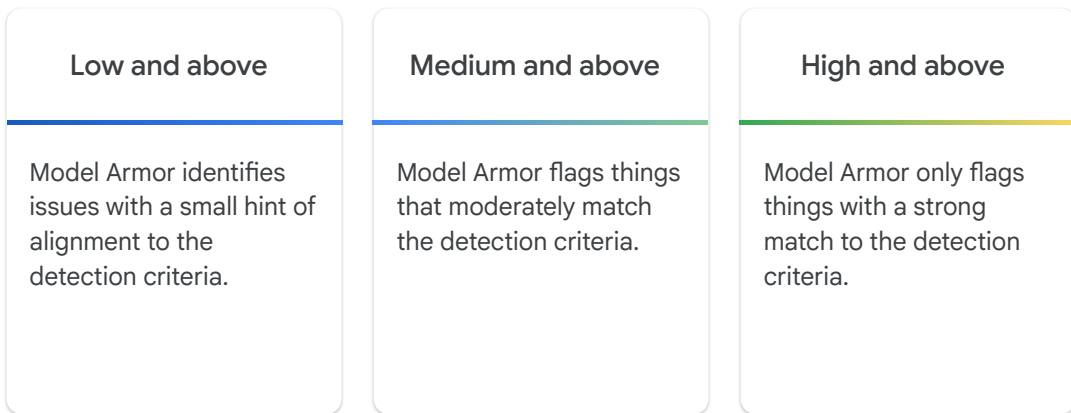
Define detection **thresholds** for flagging potential violations.

Model Armor also uses confidence levels to determine how sensitive a detection rule should be.

These levels help determine how strongly a prompt or response must match a rule before it is flagged.

For example, stricter environments may use lower thresholds to detect potential issues earlier.

## Confidence levels

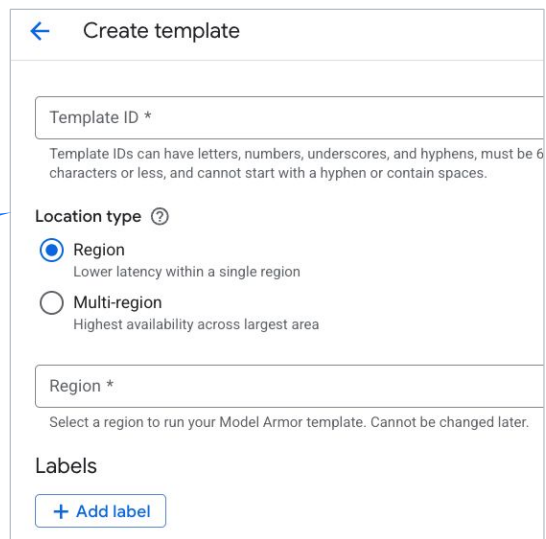


Let's look at each level in a bit more detail.

- With the low and above level, Model Armor screens almost everything. At this level, it's going to identify issues with the smallest hint of alignment to the detection criteria.
- At the medium and above level, Model Armor is a bit more discerning. It flags things that are moderate matches to the detection criteria.
- At the high and above level, Model Armor only flags things when confident that the information is a strong match to the detection criteria

## Template configuration: General info

Specify template ID,  
region and labels



← Create template

Template ID \*

Template IDs can have letters, numbers, underscores, and hyphens, must be 6 characters or less, and cannot start with a hyphen or contain spaces.

Location type ?

☒ Region  
Lower latency within a single region

☐ Multi-region  
Highest availability across largest area

Region \*

Select a region to run your Model Armor template. Cannot be changed later.

Labels

+ Add label

Templates define how Model Armor evaluates prompts and responses.

- When creating a template, you specify attributes such as template ID, region, and metadata labels.
- Templates can then be applied to different workloads or services.

At the top of the template, you'll find fields to capture the template ID and the region where you want to run the template.

Multiregional resources are redundant and distributed within and across regions.

You can also add a label to your templates so that you can group and categorize them.

## Template detections: Malicious URL, prompt injection and jailbreak detection

Protect LLM from encountering phishing sites, malware traps, and cyber attack hotspots by checking this box.

Block malicious content from entering your LLM prompts and assign a confidence score

The screenshot shows a 'Create template' window with a 'Detections' section. It contains three items: 'Malicious URL detection' (checked), 'Prompt injection and jailbreak detection' (checked), and 'Sensitive data protection' (unchecked). A 'Confidence level' dropdown is set to 'Medium and above'. A note explains that setting the confidence level to 'Low and above' would detect more content.

← Create template

**Detections**

- ☒ **Malicious URL detection**  
Identifies web addresses (URLs) that are designed to harm users or systems. These URLs might lead to phishing sites, malware downloads, or other cyberattacks.
- ☒ **Prompt injection and jailbreak detection**  
Prompt injection is when a malicious actor tries to insert malicious content into a prompt. A jailbreak attempt is when a malicious actor tries to break out of the model's safety controls. Both can make the AI ignore its usual instructions, reveal sensitive information, bypass an AI model's safeguards, or make it perform actions it wasn't designed to do.
- ☐ **Sensitive data protection**  
Detects sensitive data and helps prevent its accidental exposure from attacks like prompt injection.

Confidence level  
Medium and above

For stricter enforcement, set confidence level to Low and above. This will detect most content that is likely to be a prompt injection and/or jailbreak attempt.

Google Cloud

Let's move to the next section of a template.

Do you want your LLM to wander into phishing sites, malware traps, or other cyberattack hot spots? Didn't think so.

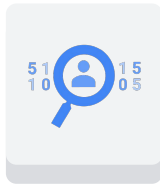
Simply check the box by malicious URL detection and you can cross that worry off your list.

Similarly for prompt injection and jailbreak detection. Check the box to block anyone from entering malicious content into your LLM prompts.

You can use to assign confidence levels to this detection.

When prompt injection and jailbreak detection is enabled, Model Armor scans prompts and responses for malicious content. If detected, Model Armor blocks the prompt or response.

# Sensitive Data Protection



Sensitive Data Protection

- ✓ Provides inspection (detection) and transformation (de-identification) capabilities for sensitive data
- ✓ Built-in support for over 240 types of information, called infoTypes, such as:
  - CREDIT\_CARD\_NUMBER
  - EMAIL\_ADDRESS
- ✓ Supports many data de-identification techniques
  - Redaction
  - Replacement
  - Masking
  - Crypto-based tokenization:
  - Date shifting
  - Etc

Sensitive Data Protection is another important security service.

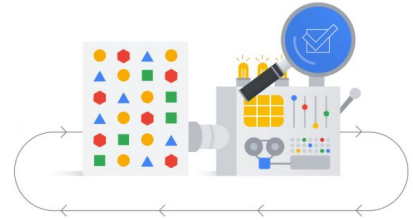
It helps identify and protect sensitive information within data streams.

The system supports detection of hundreds of information types such as credit card numbers, email addresses, and personal identifiers.

It also supports several de-identification techniques including masking, redaction, and tokenization.

# Sensitive Data Protection: DLP API

- The Data Loss Prevention (DLP) API is the programmatic interface for Sensitive Data Protection services
  - Can call the API directly without leveraging Model Armor
- Could be used to allows your agent to be surgical
  - Cleaning only a specific parts of a message that are sensitive
- Can create and use Sensitive Data Protection templates
  - Then simply call the API and say, "Use Template X"
  - Keeps the code clean and allows security teams to update rules without redeploying your agent



Sensitive Data Protection exposes its functionality through the DLP API.

Applications can call this API directly to inspect and sanitize content.

This allows agents to detect sensitive information and remove or mask it before continuing a workflow.

Security teams can also define reusable templates that standardize these rules across applications.

## Sensitive Data Protection: Sanitize data sources

- Sensitive Data Protection could also be used to sanitize data sources
  - Instead of sanitizing model responses
- For example:
  - You have a bunch of data used to create a RAG data store
  - The agent allows the user to perform actions against the data
  - If that data contains sensitive information, Model Armor can redact the responses
  - Another solution could be to redact the data before it is fed to the RAG data store
    - The model will not even have access to any sensitive data
    - Use the Sensitive Data Protection service to redact the data in the storage bucket before creating the RAG corpus

Instead of using SDP and Cloud Armor to scan responses for sensitive info and redact that information, what if we just redact the source of the data before we give it to the model.

Sensitive Data Protection can also be used before data reaches the model.

For example, when building a RAG knowledge base, sensitive information can be removed from the dataset before it is indexed.

This ensures the model never has access to sensitive information in the first place.

This approach reduces risk by preventing exposure rather than reacting to it later.

# Model Armor's Sensitive Data Protection

## Basic Sensitive Data Protection

Helps you detect a fixed set of Sensitive Data Protection infoTypes:

- CREDIT\_CARD\_NUMBER
- FINANCIAL\_ACCOUNT\_NUMBER
- GCP\_CREDENTIALS
- GCP\_API\_KEY
- PASSWORD
- US\_SOCIAL\_SECURITY\_NUMBER\*
- US\_INDIVIDUAL\_TAXPAYER\_IDENTIFICATION\_NUMBER\*

\* for US-based regions only

## Advanced Sensitive Data Protection

Can create custom Sensitive Data Protection templates

Supports built-in and custom infoTypes

Can detect, redact, mask, or tokenize data

Model Armor integrates with Sensitive Data Protection in two modes. Basic mode detects a predefined set of common sensitive data types. Advanced mode allows teams to define custom templates and detection rules. Advanced mode also supports additional transformation techniques such as tokenization or masking.

In Basic mode: the following Sensitive Data Protection infoTypes are scanned in the prompt for all regions:

**CREDIT\_CARD\_NUMBER:** A credit card number is 12 to 19 digits long. It is used for payment transactions globally.

**FINANCIAL\_ACCOUNT\_NUMBER:** A number referring to a specific financial account, for example, a bank account number or a retirement account number.

**GCP\_CREDENTIALS:** Google Cloud service account credentials. Credentials that can be used to authenticate with {api\_client\_lib\_name} and service accounts.

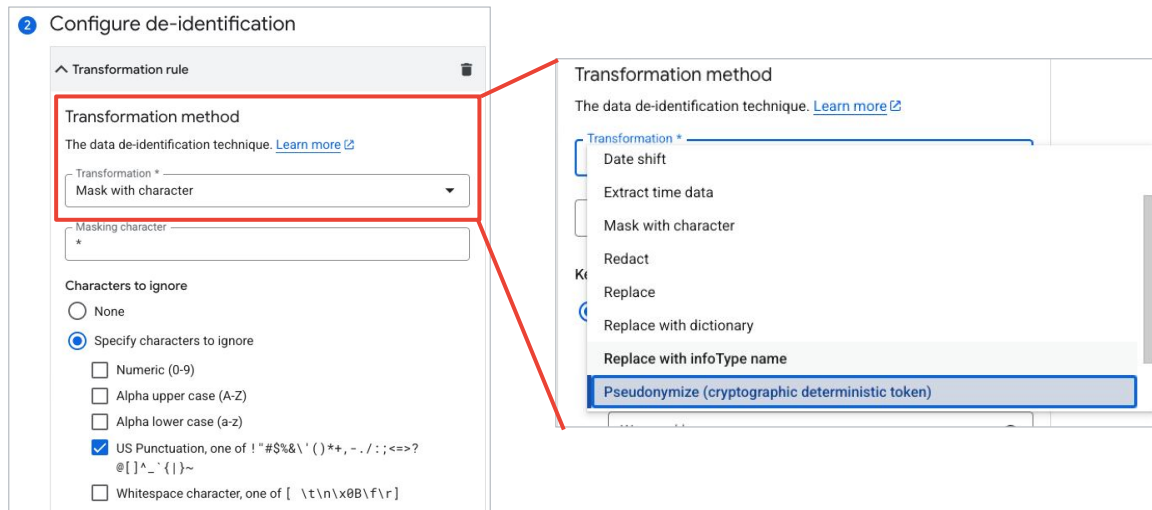
**GCP\_API\_KEY:** Google Cloud API key. An encrypted string that is used when calling Google Cloud APIs that don't need to access private user data.

**PASSWORD:** Clear text passwords in configs, code, and other text.

**US\_SOCIAL\_SECURITY\_NUMBER** and **US\_INDIVIDUAL\_TAXPAYER\_IDENTIFICATION\_NUMBER** infoTypes are scanned in the prompt for US-based

regions.

# Sensitive Data Protection templates



Google Cloud

Proprietary & Confidential 18

Sensitive Data Protection templates allow organizations to define standardized policies.

These templates can specify how sensitive data is detected and how it should be transformed.

For example, templates may define rules for masking credit card numbers or tokenizing identifiers.

# Model Armor template detections: Sensitive Data Protection

In Model Armor template,  
specify the Sensitive Data  
Protection templates to use

Can specify both inspection  
templates and de-identify  
templates

☒ **Sensitive data protection**  
Detects sensitive data and helps prevent its accidental exposure from attacks like prompt injection.

**Detection type**

☐ Basic  
Use [predefined infoTypes](#) to detect sensitive data types

☒ **Advanced**  
Use inspection template defined in sensitive data protection service as a single source for sensitive data infoTypes

**Sensitive Data Protection templates**  
Inspection templates specify what sensitive data to scan for, such as phone numbers, credit card numbers, or addresses. De-identification templates use the results to remove or redact specified content. You can create [Inspection templates](#) and [De-identification templates](#) using the Sensitive Data Protection service.

Inspect template \*

Enter the inspection template name. Specify full path in this format:  
projects/projectId/locations/locationID/inspectTemplates/templateId

De-identify template

Model Armor templates can reference Sensitive Data Protection templates.

This allows Model Armor to automatically apply data protection policies when inspecting prompts or responses.

By combining both systems, organizations can implement comprehensive security controls.

# Responsible AI (RAI)

Confidence levels tell Model Armor how sure to be when flagging violations



← Create template

### Responsible AI

Confidence level represents how likely it is that the findings match a content filter type. For stricter enforcement, set confidence level to "Low and above" to detect most content that falls into a content filter type.

Customize confidence levels for each content filter below or set confidence level for all content filters.

[Set confidence level for all content filters](#) ▾

- Hate Speech — Medium and above ▾
- Dangerous — Medium and above ▾
- Sexually Explicit — Low and above ▾
- Harassment — Medium and above ▾

Responsible AI filters are also integrated into Model Armor.

These filters detect harmful content such as hate speech, harassment, or explicit content.

Confidence levels determine how aggressively these filters detect violations.

Keep in mind that the confidence level has the floor setting guard rails as the starting point. You can have a more strict confidence level, but you can't relax the level below the floor setting minimum.

# Define the enforcement type

Enforcement defines what happens after a violation is detected.

- Model Armor by default inspects requests that violate the configured settings, but it doesn't block them
  - Your application needs to take action
- Model Armor can inspect and block violations
  - Available for Gemini models on the Agent Platform and must be enabled with a Floor setting
- Model Armor can de-identify sensitive data in the prompt and response
  - If you are using the Advanced Sensitive Data Protection

## Model Armor offers following integration modes

### Inline Mode

- Automated, infrastructure-level protection
  - No code changes
- Optimized for Google services

### Do-It-Yourself (REST API) Mode

- Application Code calls the Model Armor service
- Any LLM model can be used
  - No enforcement to use Google only LLMs
- Offers more flexibility and can operate the environment in a heterogeneous fashion
- Allows for highly customized safety logic, i.e.
  - Action for detection under 'responsible-ai' can be different from action for detection under 'prompt-injection'

Model Armor supports two integration modes.

- Inline mode provides automated protection at the infrastructure level without requiring application code changes.
- Do-It-Yourself mode allows applications to call Model Armor directly through the REST API.

DIY mode offers greater flexibility and customization.

## Model Armor inline mode

|                   | Configure detectors               | Model and cloud coverage                  | Enable the integration                                                                                                 |
|-------------------|-----------------------------------|-------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Agent Platform    | Using floor settings or templates | Gemini (non-streaming) on Google Cloud    | Can use Floor settings or templates to enforce security project-wide for all Gemini calls without any networking setup |
| GKE               | Using templates                   | Models with OpenAI format on Google Cloud | Configure a service extension on application load balancers to screen traffic to and from app                          |
| Cloud Run         | Using templates                   | Models with OpenAI format on Google Cloud | Configure a service extension on application load balancers to screen traffic to and from app                          |
| Gemini Enterprise | Using templates                   | All models and all clouds                 | Apply Model Armor policies to the Gemini Enterprise instance                                                           |

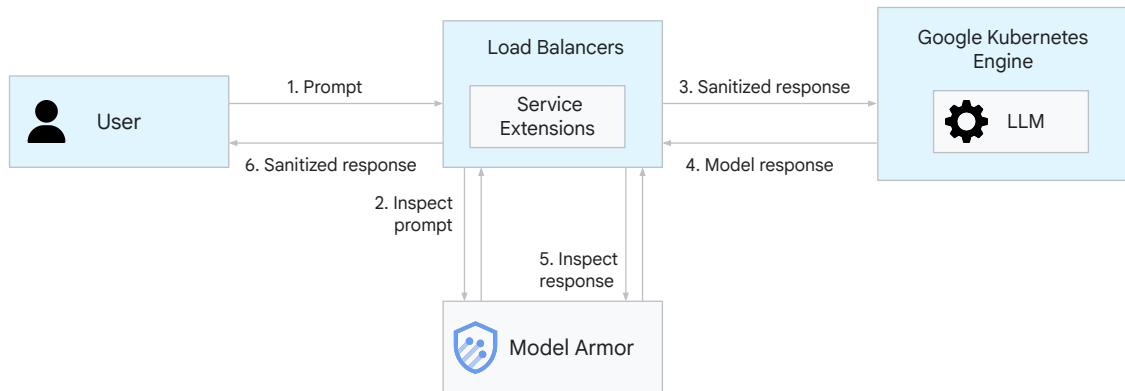
Inline mode is optimized for Google Cloud environments.

It allows organizations to enforce Model Armor policies across services such as Agent Platform, Cloud Run, or GKE.

In this mode, prompts and responses are automatically inspected without modifying application code.

<https://docs.cloud.google.com/model-armor/integrations>

## GKE inline mode: How it works



Configure a service extension on a load balancer that routes traffic to an LLM hosted in your GKE cluster. This configuration specifies that Model Armor should be used to screen prompts and responses.

This slide shows how inline mode works within a Kubernetes environment.

Traffic passes through a load balancer extension that sends prompts and responses to Model Armor for inspection.

This ensures security policies are enforced consistently across services.

Image taken from

<https://docs.cloud.google.com/model-armor/model-armor-gke-integration>

## Do-It-Yourself (REST API) mode steps

01

### Make a call

Call Model Armor to screen user prompts and model responses

02

### Read responses

Parse Model Armor's responses after each completed call

03

### Decide what to do

Think about the best way to respond to what Model Armor provided

In DIY mode, the application directly calls Model Armor.

The workflow typically includes three steps.

- First, send prompts or responses to Model Armor.
- Second, inspect the results returned by the service.
- Third, decide how the application should respond to any detected issues.

## Responding to Model Armor

Here are some decisions you can make when responding to what Model Armor discovered:

- ✓ Block the user's request
- ✓ Block the model's response
- ✓ Warn the user
- ✓ Use redacted text and continue workflow

Applications can respond to Model Armor results in several ways.

- They may block the request entirely.
- They may sanitize or redact sensitive data.
- They may warn the user about a potential issue.
- Or they may continue processing with modified content.

## DIY mode: sanitize user prompts example (Python)

```
from google.cloud import modelarmor_v1

PROMPT_INJECTION_TEMPLATE="projects/my-project/locations/us/templates/check-prompts-template"

client = modelarmor_v1.ModelArmorClient(transport="rest", client_options = {"api_endpoint" :
"modelarmor.us.rep.googleapis.com"})

# Function to check user prompts for invalid requests.
def sanitize_prompt(user_prompt):
    user_prompt_data = modelarmor_v1.DataItem()
    user_prompt_data.text = user_prompt
    request = modelarmor_v1.SanitizeUserPromptRequest(
        name=f"{PROMPT_INJECTION_TEMPLATE}",
        user_prompt_data=user_prompt_data,
    )
    response = client.sanitize_user_prompt(request=request)

    # Returns FilterMatchState.NO_MATCH_FOUND (1) or FilterMatchState.MATCH_FOUND (2)
    return response.sanitization_result.filter_match_state
```

Here's a simple Python example that shows the full workflow, from calling Model Armor to deciding what to do next.

## DIY mode: sanitize model responses example (Python)

```
RESPONSE_TEMPLATE="projects/my-project/locations/us/templates/check-response-template"

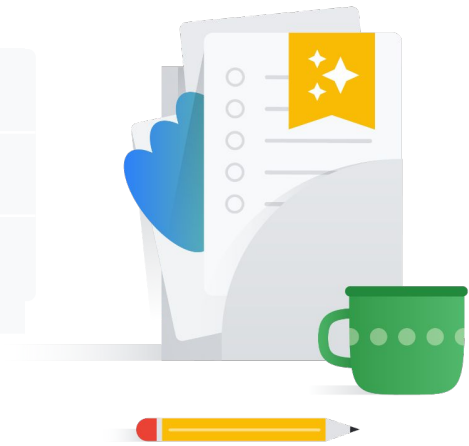
def sanitize_response(response_text):
    model_response_data = modelarmor_v1.DataItem()
    model_response_data.text = response_text
    request = modelarmor_v1.SanitizeModelResponseRequest(
        name=f"{RESPONSE_TEMPLATE}",
        model_response_data=model_response_data,
    )
    response = client.sanitize_model_response(request=request)

    # Return the Sanitized text if sensitive data was found.
    if response.sanitization_result.filter_match_state == FilterMatchState.MATCH_FOUND:
        sanitized_text =
            response.sanitization_result.filter_results["sdp"].
            sdp_filter_result.deidentify_result.data.text
        return sanitized_text
    else:
        # There was no invalid data, so just return what was sent in
        return response_text
```

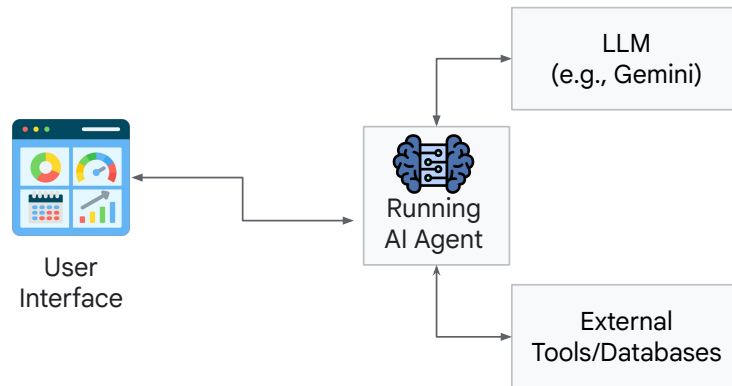
Here's a simple Python example that shows the full workflow, from calling Model Armor to deciding what to do next.

# Topics

- |    |                                                                           |
|----|---------------------------------------------------------------------------|
| 01 | Model and context security                                                |
| 02 | <a href="#">Agent access</a>                                              |
| 03 | Lab: Enhancing AI Security with Model Armor and Sensitive Data Protection |



# Architecture of an agent-enabled application



An agent-enabled application typically includes several components.

The user interface captures requests.

The agent orchestrates logic and tool usage.

The LLM performs reasoning.

External tools and databases provide additional capabilities.

Each component introduces potential security considerations.

## Agent success: Securing user-to-agent connection

| Component      | Responsibility                                                                        | Key Security Consideration                                   |
|----------------|---------------------------------------------------------------------------------------|--------------------------------------------------------------|
| User Interface | Captures intent and displays final reasoning so the user sees what the agent is doing | <b>End-user Authentication</b><br>(Identity Platform)        |
| Agent          | The backend logic that manages state and flow                                         | <b>Identity and Access Management (IAM)</b> service accounts |
| LLM            | Reasoning and language understanding                                                  | <b>Data Residency</b> and <b>Safety Filters</b>              |
| Tools/APIs     | Backend APIs or services the agent can call (BigQuery, Search, ERP)                   | <b>OAuth Scopes</b> and <b>Least Privilege</b> access        |

Different components in the architecture have different security responsibilities.

User interfaces must authenticate users.

Agents should run with controlled identities using service accounts.

LLMs require safety filters and data residency considerations.

External tools should use least-privilege access policies.

## Securing the connection between a user and an agent

### Identity Aware Proxy (IAP) Pattern

- Managed Google Cloud service that authenticates the user
- Protects user → agent HTTP ingress



IAP Pattern

### Backend-for-Frontend (BFF) Pattern

- A custom proxy authenticates the user and forwards requests
- Translates user identity to workload identity



BFF Pattern

### Workload Identity Federation (WIF) Pattern

- Allows external applications to impersonate a Google Cloud service account
- Federates external workload identity into Google Cloud



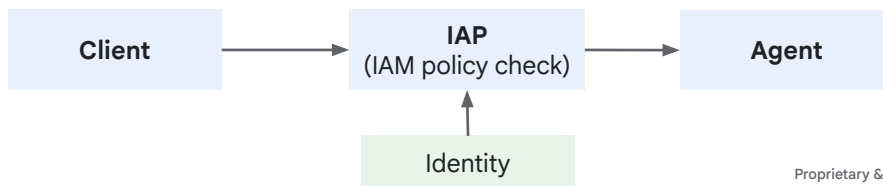
WIF Pattern

Three common patterns are used to secure agent access.

1. The first is Identity-Aware Proxy, which protects HTTP endpoints.
2. The second is the Backend-for-Frontend pattern, where a proxy handles authentication and authorization.
3. The third is Workload Identity Federation, which allows external systems to securely access Google Cloud resources.

# Identity-Aware Proxy (IAP) for Agent HTTP Boundary Protection

- Authenticates users via Google Identity or external IdP (OIDC)
  - Enforces IAM policies (roles/iap.httpsResourceAccessor) before request reaches backend
  - Injects **X-Goog-IAP-JWT-Assertion** header (signed JWT)
  - Blocks unauthenticated/unauthorized requests
- For Agent Workloads (Cloud Run / GKE / App Engine)
  - Protects the agent's HTTP ingress
  - Removes auth logic from agent code
    - Backend should validate IAP-signed JWT header for zero-trust verification
  - Backend must not allow unauthenticated direct invocation



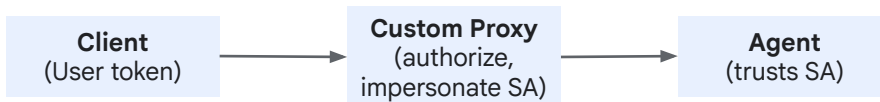
Identity-Aware Proxy authenticates users before requests reach the agent backend.

It enforces IAM policies and injects identity headers into the request.

This allows the backend to trust that the user has already been authenticated.

## Securing agent endpoints with BFF

- Application-layer security and identity translation for agent access
  - Authenticates end-users (OIDC, Firebase Auth, Auth0, etc.)
  - Validates and verifies user ID tokens
  - Maps user identity → Google Cloud service account
    - User identity terminates at the BFF
  - Calls the agent using service account (SA) credentials
    - Agent trusts Google Cloud Service Account (impersonation or Workload Identity)
- Agent endpoint is never publicly exposed
  - Agent only trusts Google Cloud service identities
  - Centralized policy enforcement and request validation
  - Enables per-user access control and multi-tenant isolation



In the Backend-for-Frontend pattern, a custom proxy handles user authentication.

The proxy validates user tokens and maps the user identity to a service account.

The agent trusts the service account identity rather than the raw user identity.

This pattern allows centralized policy enforcement.

## Securing agent endpoints with WIF

- Secure service-to-service and cross-cloud agent access without static keys
  - External workload obtains OIDC/SAML token
  - Google STS exchanges token for federated access token
  - Optional service account impersonation
  - Agent/API validates Google-issued token
  - Not for browser-based user access
- Eliminates long-lived service account keys
  - Short-lived credentials ( $\leq 1$  hour)
  - Explicit, scoped trust configuration
  - Enables least-privilege cross-cloud access



Workload Identity Federation allows external systems to authenticate to Google Cloud without using static credentials.

Instead, external identity tokens are exchanged for short-lived Google access tokens.

This reduces the risk of credential leakage.

## IAP - BFF - WIF comparison

| Feature                     | IAP                          | BFF                                                 | WIF                                   |
|-----------------------------|------------------------------|-----------------------------------------------------|---------------------------------------|
| Primary use case            | Protect HTTP endpoint        | Enforce business policy + identity translation      | Federate external workload identity   |
| Protects traffic type       | User → agent (HTTP ingress)  | User → agent (with application-layer authorization) | Workload → agent (service-to-service) |
| Requires custom code        | No                           | Yes                                                 | No                                    |
| Eliminates static keys      | N/A                          | Optional                                            | Yes                                   |
| Where auth logic lives      | Google-managed control plane | Custom application layer                            | Google STS + IAM control plane        |
| Identity presented to agent | End-user identity (IAP JWT)  | Google service account (impersonated)               | Google-issued federated access token  |

Each pattern serves a different purpose.

- IAP protects HTTP ingress for user requests.
- BFF enables custom authorization logic.
- WIF enables secure service-to-service communication across environments.

Organizations often combine these patterns to create a layered security architecture.

# SPIFFE for Agent Workload Identity

- SPIFFE: Open standard for cryptographic workload identity in zero-trust environments
- Identity lifecycle
  - Workload attests to SPIRE (identity authority)
  - SPIRE issues short-lived SVID (X.509 or JWT)
  - Workloads mutually authenticate via mTLS using SPIFFE-issued identities
  - Authorization decisions use SPIFFE identity
    - ID format: `spiffe://trust-domain/workload-name`
  - Authorization policies evaluate SPIFFE identity
- SPIFFE is cloud-agnostic and independent of cloud IAM systems
  - SPIFFE authenticates workloads
  - IAM authorizes access to Google Cloud resources

SPIFFE = Secure Production Identity Framework for Everyone  
SPIRE = SPIFFE Runtime Environment

SPIFFE is an open standard for workload identity. It provides cryptographic identities for services in distributed systems. These identities are typically used with mutual TLS to authenticate workloads. SPIFFE is cloud-agnostic and complements IAM-based authorization. SPIFFE **does not replace IAM**—it complements it.

## How SPIFFE Works (Conceptually)

1. A workload (service, container, agent) starts.
2. It proves its identity to an identity authority (SPIRE).
3. SPIRE issues a short-lived credential called an SVID.
4. Services authenticate to each other using mutual TLS (mTLS) and their SPIFFE identities.

## Agent Identity: Service accounts vs SPIFFE

| Dimension             | Service Accounts              | SPIFFE                                  |
|-----------------------|-------------------------------|-----------------------------------------|
| Identity Type         | Cloud IAM identity            | Cryptographic workload identity         |
| Issued By             | Google Cloud IAM              | SPIRE (identity authority)              |
| Credential Form       | OAuth2 access token           | X.509 SVID or JWT-SVID                  |
| Primary Purpose       | Access Google Cloud resources | Mutual authentication between workloads |
| Scope                 | Google Cloud-native           | Cloud-agnostic                          |
| Trust Model           | IAM policy-based              | mTLS + cryptographic identity           |
| Rotates Automatically | Yes (token-based)             | Yes (short-lived certs)                 |
| Replaces IAM?         | N/A                           | No                                      |

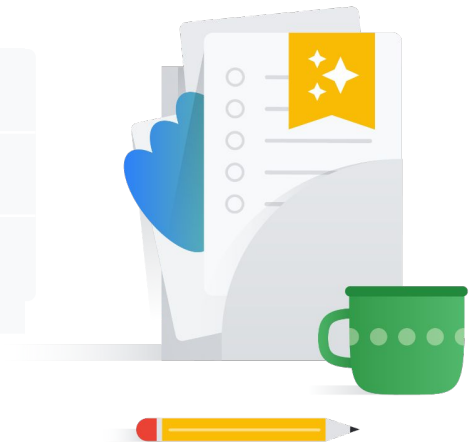
Service accounts provide identity within Google Cloud IAM.

SPIFFE provides cryptographic workload identity independent of cloud providers.

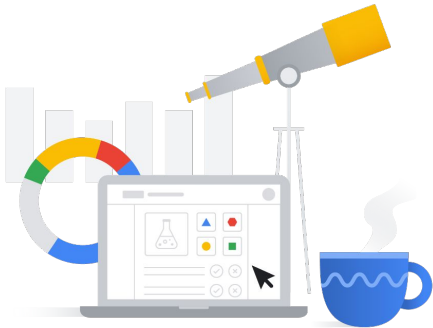
Service accounts control access to Google Cloud resources, while SPIFFE enables secure communication between workloads.

# Topics

- |    |                                                                                           |
|----|-------------------------------------------------------------------------------------------|
| 01 | Model and context security                                                                |
| 02 | Agent access                                                                              |
| 03 | <a href="#">Lab: Enhancing AI Security with Model Armor and Sensitive Data Protection</a> |



# Lab: Enhancing AI Security with Model Armor and Sensitive Data Protection



01

Create a RAG data store

02

Evaluate model response for safety

03

Create Model Armor and Sensitive Data Protection templates

04

Protect PII using Model Armor integrated with Sensitive Data Protection

In this lab, you configure Model Armor and Sensitive Data Protection.

You will create a RAG data store, evaluate responses for safety, create security templates, and apply protections to prevent sensitive data exposure.

# Questions and answers





Thank you.  
Module 5 is complete.